

Contents

Python Cheat Sheet	2
1. Datentypen, Typisierung & Built-ins	2
Wichtige Eigenschaften	2
Wichtige Built-ins & Methoden	2
String-Methoden (Erzeugen immer einen neuen String!)	2
F-Strings	2
Listen-Methoden (Modifizieren die Liste meist in-place!)	3
Set-Operationen (Mengenlehre)	3
Typisierungskonzepte	3
2. Ablaufsteuerung & Algorithmen	3
KLAUSURFALLE: Mutable Default-Parameter	3
Rekursion	3
Slicing, Comprehensions & Match-Case	3
3. Funktionen, Exceptions & Dateihandling	4
Funktionale Werkzeuge & Generatoren	4
Exceptions (try-except-else-finally)	4
Kontextmanager & CSV-Handling	4
4. Objektorientierte Programmierung (OOP)	4
Klassen, Sichtbarkeiten & UML	4
5. Testing & Binärdaten	5
Embedded C Cheat Sheet	6
1. Build-Prozess & C-Toolchain	6
2. Speicher-Layout, Variablen & Pointer	6
Speicherbereiche (Memory Layout)	6
Wichtige Speicherklassen (Storage Classes)	6
Pointer & Call-by-Reference	6
Zuweisungen bei Pointern (Klausurfalle!)	7
Wild Pointer, Dangling Pointer & Segmentation Fault	7
3. Structs, Alignment, Padding & Endianness	8
Speicherausrichtung (Alignment) & Padding	8
Padding	8
Speichermanipulation	8
Alignment	8
Bitfelder (Bitfields)	8
Endianness (Byte-Reihenfolge)	8
4. Hardwarenahe Programmierung	9
Direkter Register-Zugriff & volatile	9
Bitmanipulation (Bitmasking)	9
5. Präprozessor & Makros	9
6. Interrupts, Nebenläufigkeit & RTOS	9
Interrupt Service Routinen (ISR)	9
RTOS, Semaphoren & Mutexe	9
7. Watchdog Timer (WDT)	10
8. Wichtige Standard-Funktionen (I/O)	10
printf() & Formatierung	10
Code Beispiele	11
Pointer	11
Makefile	11
Linker Skripte	12
Linked Lists	14

Python Cheat Sheet

Prüfungsrelevante Syntax & Konzepte

1. Datentypen, Typisierung & Built-ins

■ Wichtige Eigenschaften

- **Mutable (Veränderbar):** Objekt kann in-place modifiziert werden (gleiche Speicheradresse).
- **Iterable:** Kann in einer Schleife durchlaufen werden (`__iter__`).

Typ	Mutable	Iterable	Beispiele / Notes
int / float / bool	Nein	Nein	42, 3.14, True
NoneType	Nein	Nein	Repräsentiert Leere / Nullwert
list	Ja	Ja	[1, "a", [2]]
tuple	Nein	Ja	(1, 2) (kann mutable Elemente enthalten!)
str	Nein	Ja	"Text"
range	Nein	Ja	range(start, stop, step) (stop ist exklusiv)
dict	Ja	Ja (Keys)	{"id": 1} -> Keys MÜSSEN immutable sein!
set	Ja	Ja	{1, 2} -> Elemente MÜSSEN immutable sein!

■ Wichtige Built-ins & Methoden

- `sum(iter, start=0)`: Summiert alle numerischen Elemente.
- `enumerate(iter)`: Liefert (Index, Wert).
- `zip(*iters)`: Aggregiert parallel zu Tupeln. Bricht ab, wenn kürzestes Iterable zu Ende.
- `sorted(iter, reverse=True)`: Gibt **neue** sortierte Liste zurück (`list.sort()` modifiziert in-place).
- **F-Strings (Float-Formatierung):** `f"Wert: {pi:.2f}"` -> 3.14 (Rundet auf 2 Nachkommastellen).

■ String-Methoden (Erzeugen immer einen neuen String!)

- `.strip()`: Entfernt Whitespaces am Anfang und Ende.
- `.split(sep)`: Zerlegt den String am Trennzeichen `sep` in eine Liste.
- `sep.join(iter)`: Verbindet eine Liste von Strings mit dem Trennzeichen `sep`.
- `.replace(old, new)`: Ersetzt Vorkommen von `old` durch `new`.
- `.lower()` / `.upper()`: Transformiert in Klein- bzw. Großbuchstaben.
- `.startswith(prefix)` / `.endswith(suffix)`: Liefert True oder False.

■ F-Strings

In Python entfällt die Puffer-Verwaltung. Strings werden dynamisch im Speicher erzeugt. F-Strings (Präfix `f`) werten Ausdrücke in den geschweiften Klammern `{}` direkt zur Laufzeit aus.

Wichtige Format-Specifier (Klausur-Klassiker):

- `:.2f`: Fließkommazahl (float) auf 2 Nachkommastellen runden.
- `:04X`: Hexadezimalwert (großgeschrieben), mit Nullen auf 4 Stellen aufgefüllt.
- `:08d`: Integer, mit führenden Nullen auf 8 Stellen aufgefüllt.
- `<10` / `>10` / `^10`: Text linksbündig / rechtsbündig / zentriert auf 10 Zeichen Breite ausrichten (aufgefüllt mit Leerzeichen).

```
messwert = 1024
spannung = 3.3456
reg_val = 255
```

```
# F-String Syntax: f"Text {variable:formatierung}"
ausgabe = f"Wert: {messwert:05d}, U: {spannung:.1f}V"
# -> "Wert: 01024, U: 3.3V"
```

```
hex_ausgabe = f"Register: 0x{reg_val:04X}"
# -> "Register: 0x00FF"
```

```
ausrichtung = f"|{messwert:<6}|{messwert:>6}|"
# -> "|1024 | 1024|"
```

■ Listen-Methoden (Modifizieren die Liste meist in-place!)

- `.append(x)`: Hängt das Element `x` hinten an die Liste an.
- `.extend(iter)`: Hängt alle Elemente des Iterables `iter` an die Liste an.
- `.insert(index, x)`: Fügt `x` an der Position `index` ein.
- `.pop(index=-1)`: Entfernt das Element am `index` (Standard: letztes) und gibt es zurück.
- `.remove(x)`: Entfernt das **erste** Vorkommen von `x` (wirft `ValueError`, wenn nicht existent).
- `.clear()`: Entfernt alle Elemente aus der Liste.
- `.sort(key=None, reverse=False)`: Sortiert die Original-Liste (Gegensatz zu `sorted()`).
- `.reverse()`: Reverse the list.

■ Set-Operationen (Mengenlehre)

Sets filtern Duplikate automatisch heraus. Logische Verknüpfungen (Bsp: `a = {1, 2}`, `b = {2, 3}`):

- **Vereinigung (Union)**: `a | b -> {1, 2, 3}` (Elemente aus `a` oder `b`)
- **Schnittmenge (Intersection)**: `a & b -> {2}` (Elemente, die in `a` UND `b` sind)
- **Differenz (Difference)**: `a - b -> {1}` (Elemente in `a`, die NICHT in `b` sind)
- **Sym. Differenz (XOR)**: `a ^ b -> {1, 3}` (Elemente, die in genau einem der Sets sind)

■ Typisierungskonzepte

- **Duck Typing**: Typ egal, Hauptsache Objekt hat benötigte Methode ("quakt wie eine Ente").
- **EAFP vs. LBYL**: EAFP (einfach ausführen, Fehler mit `try/except` fangen) ist Python-Standard. LBYL (vorher mit `if` prüfen) begünstigt Race Conditions.

2. Ablaufsteuerung & Algorithmen

■ KLAUSURFALLE: Mutable Default-Parameter

Default-Argumente werden nur **einmalig** bei Funktionsdefinition ausgewertet. Niemals mutbare Typen (wie `[]`) als Default setzen!

```
# FALSCH: Teilt die Liste über alle Aufrufe hinweg!
def append_to(element, target=[]):
    target.append(element)
```

```
# RICHTIG:
def append_to_clean(element, target=None):
    if target is None: target = []
    target.append(element)
```

■ Rekursion

Zwei zwingende Komponenten für jede rekursive Funktion:

1. **Basisfall**: Abbruchbedingung (z.B. `if n <= 1: return 1`), verhindert Endlosschleife.
2. **Rekursionsschritt**: Selbstaufruf mit modifiziertem, verkleinertem Problem.

■ Slicing, Comprehensions & Match-Case

- **Slicing**: `sequence[start:stop:step]` (`stop` ist exklusiv!). Invertieren: `liste[::-1]`
- **List Comprehension**: `[expr for item in iter if cond]`
- **Match-Case (ab 3.10)**:

```
match kommando:
    case "start" | "run": return "Startet" # OR-Verknüpfung
    case ["move", direction]: return direction # Sequenz-Entpacken
    case _: return "Default / Wildcard"
```

3. Funktionen, Exceptions & Dateihandling

■ Funktionale Werkzeuge & Generatoren

- **Generatoren:** Nutzen `yield` statt `return`. Erlauben träge Auswertung (**Lazy Evaluation**). Berechnen Werte erst bei Bedarf (spart RAM).
- **Lambda:** Anonyme Einzeiler: `lambda x, y: x + y`
- `map(func, iter)`: Wendet Funktion auf alle Elemente an.
- `filter(cond, iter)`: Behält nur Elemente, bei denen die Bedingung `True` liefert.
- `reduce(func, iter)`: Reduziert Iterable schrittweise auf Endwert (braucht `functools`).
- `*args` / `**kwargs`: Nimmt variable Argumente als **Tupel** (`args`) bzw. **Dict** (`kwargs`).

■ Exceptions (try-except-else-finally)

```
try:
    wert = int("abc") # Wirft ValueError
except ValueError as e:
    print("Falscher Typ!")
except Exception as e:
    print("Fängt alle restlichen Fehler")
else:
    print("Läuft NUR, wenn KEIN Fehler auftrat")
finally:
    print("Läuft IMMER (gut für Aufräumarbeiten/close)")
```

■ Kontextmanager & CSV-Handling

```
# CSV Schreiben (ohne csv-Modul)
daten = [{"ID", "Name"}, [1, "Alice"], [2, "Bob"]]

with open("daten.csv", "w", encoding="utf-8") as f:
    for zeile in daten:
        # Alle Elemente in Strings umwandeln und mit Semikolon verbinden
        zeile_string = ";".join(str(element) for element in zeile)
        f.write(zeile_string + "\n")

# CSV Lesen (ohne csv-Modul)
with open("daten.csv", "r", encoding="utf-8") as f:
    for zeile in f:
        # Zeilenumbruch (.strip) entfernen und am Semikolon trennen
        werte = zeile.strip().split(";")
        print(werte)
```

4. Objektorientierte Programmierung (OOP)

■ Klassen, Sichtbarkeiten & UML

- **Public (name):** Überall les/schreibbar (UML: +)
- **Protected (_name):** Nur Konvention, nicht strikt (UML: #)
- **Private (__name):** Aktiviert **Name Mangling** -> wird intern zu `_Klasse__name`. Direkter Zugriff von außen wirft `AttributeError`! (UML: -)
- **UML-Beziehungen:** Leere, geschlossene Pfeilspitze = **Vererbung**. Einfache Linie = **Assoziation**.

```
from abc import ABC, abstractmethod

class Konto(ABC): # Abstrakte Basisklasse
    bank_name = "Sparkasse" # Klassenattribut (statisch, geteilt von allen)

    def __init__(self, inhaber):
        self.inhaber = inhaber # Public (+)
        self.__saldo = 0 # Private (-)

    @property
    def saldo(self): # GETTER (Aufruf ohne Klammern: k.saldo)
        return self.__saldo
```

```

@saldo.setter
def saldo(self, wert):          # SETTER (Zuweisung: k.saldo = 100)
    if wert < 0: raise ValueError()
    self.__saldo = wert

@abstractmethod
def sprich(self): pass          # MUSS in Subklasse überschrieben werden

@classmethod
def von_string(cls, string): # Factory (erhält Klasse als 'cls')
    return cls(string)

@staticmethod
def info():                     # Braucht weder self noch cls
    return "Bank-Info"

```

5. Testing & Binärdaten

- **TDD Zyklus:** Red (Test schlägt fehl) -> Green (Test klappt) -> Refactor (Code aufräumen).
- **Pytest Error Check:** with `pytest.raises(TypeError)`:
- **Binärdateien:** Modus `rb` (Lesen) oder `wb` (Schreiben).
 - bytes: **Immutable** (`b"Hello"`)
 - bytearray: **Mutable** (`bytearray(b"Hello")`)

Embedded C Cheat Sheet

Prüfungsrelevante Syntax, Speicher & Hardware

1. Build-Prozess & C-Toolchain

Werkzeug	Eingabeformat	Ausgabeformat
C-Präprozessor	C-Code (.c, .h)	Erweiterter C-Code (Makros aufgelöst, Includes eingefügt)
C-Compiler	Erweiterter C-Code	Assemblercode (.s oder .asm)
Assembler	Assemblercode	Maschinencode / Object-Code (.o)
Linker	Maschinencode (.o + Libs)	Ausführbare Datei (.elf, .bin, .hex) + Map-File (.map)

Hinweis: Das .map-File des Linkers ist essenziell, um zu prüfen, wie viel RAM/Flash verbraucht wird und wo Variablen im Speicher exakt liegen.

2. Speicher-Layout, Variablen & Pointer

Ein kompiliertes C-Programm wird im Speicher in spezifische Segmente unterteilt:

■ Speicherbereiche (Memory Layout)

- `.text` / `.rodata` (Flash/ROM): Speichert ausführbaren Maschinencode (`.text`) und Konstanten/Strings (`.rodata`). Read-Only!
- `.data` (RAM): Speichert initialisierte globale/statische Variablen (z.B. `int x = 5;`). Initialwerte werden beim Booten vom Flash ins RAM kopiert.
- `.bss` (RAM): Speichert uninitialisierte globale/statische Variablen (z.B. `int y;`). Wird vom Startup-Code mit 0 initialisiert.
- Heap (RAM): Wächst "nach oben". Für dynamischen Speicher (`malloc`, `free`). Bei Embedded oft vermieden (Fragmentierungsgefahr!).
- Stack (RAM): Wächst "nach unten". Für lokale Variablen, Funktionsparameter, Rücksprungradressen. Automatische Verwaltung. Gefahr: Stack Overflow.

Keywords:

- `volatile`: Verbieta dem Compiler Optimierungen und Caching. Zwingend für Hardware-Register, ISR-Variablen und geteilte RTOS-Daten.
- `static`: Lokal: Behält den Wert zwischen Aufrufen (Speicherung im RAM, nicht Stack). Global: Begrenzt Sichtbarkeit strikt auf die aktuelle `.c`-Datei.
- `const`: Schreibschutz. Legt Daten platzsparend im Flash (`.rodata`) statt im RAM ab. (Klausurfalle: `const volatile` für Read-Only HW-Register).
- `extern`: Deklariert Variablen/Funktionen aus anderen `.c`-Dateien. Der Linker verbindet die Adressen später.
- `inline`: Fügt den Funktionscode direkt an der Aufrufstelle ein, um den Performance-Overhead eines echten Funktionsaufrufs zu sparen.

■ Wichtige Speicherklassen (Storage Classes)

`static`:

Lokale Variable: Behält ihren Wert zwischen Funktionsaufrufen (wird in `.data/.bss` statt auf dem Stack gespeichert).

Globale Variable / Funktion: Begrenzt die Sichtbarkeit strikt auf die aktuelle `.c`-Datei (Kapselung).

`const`: Deklariert Werte als unveränderlich. Der Compiler legt diese meist ins `.rodata`-Segment (spart wertvolles RAM).

`extern`: Deklariert eine Variable, die in einer anderen `.c`-Datei definiert wurde.

■ Pointer & Call-by-Reference

Pointer speichern Speicheradressen. Wichtig, um große Structs effizient zu übergeben oder mehrere Rückgabewerte zu erzeugen.

`&variable`: "Adresse-von" Operator.

`*pointer`: Dereferenzierungs-Operator.

```
int div_int(int a, int b, int *res) {
if (b == 0) return 1; // Fehlercode
*res = a / b;        // Wert an die übergebene Adresse schreiben
return 0;            // SUCCESS
}
```

■ Zuweisungen bei Pointern (Klausurfalle!)

```
struct data *old = malloc(sizeof(struct data));
struct data *new = malloc(sizeof(struct data));
```

old = new; // FALSCH! Pointer wird umgebogen. Speicher von 'old' ist verloren (Leak)!

*old = *new; // RICHTIG! Kopiert den tatsächlichen INHALT von 'new' nach 'old'.

■ Wild Pointer, Dangling Pointer & Segmentation Fault

• Wild Pointer (Wilder Zeiger):

Ein Pointer, der deklariert, aber nicht initialisiert wurde. Er enthält einen zufälligen "Müllwert" aus dem Speicher als Adresse. Das Dereferenzieren eines Wild Pointers führt zu völlig unvorhersehbarem Verhalten oder sofortigen Abstürzen.

Gegenmaßnahme: Pointer bei der Deklaration immer direkt mit NULL oder einer gültigen Adresse initialisieren (int *ptr = NULL;).

• Dangling Pointer (Hängender Zeiger):

Ein Zeiger, der auf eine Speicheradresse zeigt, die nicht mehr gültig ist oder bereits freigegeben wurde. Der Pointer "hängt" quasi in der Luft.

Ursachen: 1. Freigabe von Heap-Speicher via free(ptr);, ohne den Pointer danach auf NULL zu setzen. Der Pointer zeigt weiterhin auf die alte Adresse, obwohl der Speicher vom System neu vergeben werden kann.

2. Rückgabe der Adresse einer lokalen Variable aus einer Funktion (nach dem Verlassen des Stack-Frames ist die Adresse ungültig -> siehe Aufgabe 12).

Gegenmaßnahme: Nach einem free(ptr); den Zeiger sofort auf NULL setzen (ptr = NULL;).

• Segmentation Fault (Speicherzugriffsfehler / SIGSEGV):

Ein Laufzeitfehler, der auftritt, wenn ein Programm versucht, auf einen Speicherbereich zuzugreifen, für den es keine Berechtigung besitzt oder der physikalisch nicht existiert.

Typische Auslöser in Klausuren:

Dereferenzieren eines NULL-Pointers.

Zugriff auf Adressen außerhalb des erlaubten Adressraums des Mikrocontrollers (siehe Aufgabe 5).

Schreibversuche in Read-Only-Segmente (z. B. Ändern eines Strings im .rodata-Segment).

Ein Pufferüberlauf (Buffer Overflow), der kritische Stack-Bereiche oder Rücksprungadressen überschreibt (siehe Aufgabe 13).

```
// 1. Wild Pointer
int *wild_ptr;
// *wild_ptr = 42; // KLAUSURFALLE: undefiniertes Verhalten / Absturz!
```

```
// 2. Dangling Pointer durch free()
int dangling_ptr = (int)malloc(sizeof(int));
*dangling_ptr = 100;
free(dangling_ptr); // Speicher ist frei, ptr zeigt immer noch hin
// *dangling_ptr = 50; // FALSCH: Speicher gehört uns nicht mehr!
```

```
// Sicheres Pattern:
dangling_ptr = NULL;
```

```
// 3. Segmentation Fault Auslöser
int *null_ptr = NULL;
// *null_ptr = 10; // Absturz: Dereferenzierung von NULL
```

3. Structs, Alignment, Padding & Endianness

■ Speicherausrichtung (Alignment) & Padding

Padding

Variablen müssen an Adressen abgelegt werden, die ein Vielfaches ihrer Größe sind (z.B. 32-Bit-int an durch 4 teilbaren Adressen). Der Compiler fügt unsichtbare "Füllbytes" (Padding) ein, um dies zu gewährleisten.

```
struct bad_layout {
char c1;      // 1 Byte (+ 3 Bytes Padding)
int i;        // 4 Bytes (Muss auf 4-Byte-Grenze liegen)
char c2;      // 1 Byte (+ 3 Bytes Padding am ENDE für Array-Alignment)
}; // sizeof = 12 Bytes!
```

```
struct good_layout {
int i;        // 4 Bytes
char c1;      // 1 Byte
char c2;      // 1 Byte
// (+ 2 Bytes Padding am Ende)
}; // sizeof = 8 Bytes! (Nach Größe absteigend sortieren spart RAM!)
```

Um das Padding zu deaktivieren kann das attribute packed verwendet werden:

```
struct layout {
char c1;      // 1 Byte (+ 3 Bytes Padding)
int i;        // 4 Bytes (Muss auf 4-Byte-Grenze liegen)
char c2;      // 1 Byte (+ 3 Bytes Padding am ENDE für Array-Alignment)
}__attribute__((packed)); // sizeof = 6 Bytes!
```

Speichermanipulation

- **memset:** Setzen aller Bytes eines Speicherbereichs auf einen bestimmten Wert

```
void*memset(void*ptr, intvalue, size_tnum);
```

- **memcpy, memmove:** Kopieren eines Speicherbereichs

```
void*memcpy(void*destination, constvoid*source, size_tnum);
```

memcpy und memmove haben die gleiche Signatur und verhalten sich (bei korrekter Verwendung gleich).

Unterschied: Bei memmove dürfen die Speicherbereiche überlappen bei memcpy nicht.

- **memcmp:** Byteweises vergleichen zweier Speicherbereicheint

```
memcmp(const void *ptr1, const void *ptr2, size_t num); Rückgabewert:<0:
```

Das erste verschiedene Byte hat in Bereich 1 (ptr1) einen kleineren Wert als in Bereich 2 (ptr2)>0: Das erste verschiedene

Byte hat in Bereich 1 (ptr1) einen größeren Wert als in Bereich 2 (ptr2) 0: Die Speicherbereiche haben identische Werte

KLAUSURFALLE memcmp(): Da Padding-Bytes zufälligen Müll enthalten, schlägt der direkte Vergleich zweier Structs mit memcmp() oft fehl. Immer die Elemente einzeln vergleichen!

Alignment

Um das Alignment an Adresse 0xAB001000 zu berechnen muss man die größte 2er Potenz finden durch die die Adresse restlos teilbar ist: da $\frac{0xAB001000}{0x1000}$ ein Integer ist, ist das Maximale Alignment 2^{12} Byte also 4 kibi Byte

■ Bitfelder (Bitfields)

Erlauben das Packen mehrerer kleiner Werte in ein einzelnes Byte/Wort. Sehr nützlich für Protokoll-Header. Achtung: Das Layout im Speicher (MSB vs. LSB) ist stark compilerabhängig!

```
typedef struct {
uint8_t flag1 : 1; // Belegt exakt 1 Bit
uint8_t flag2 : 1;
uint8_t mode : 6; // Belegt 6 Bits
} status_reg_t; // sizeof = 1 Byte
```

■ Endianness (Byte-Reihenfolge)

Speicherung von Mehrbyte-Variablen (z.B. 0xAABBCCDD) im Speicher:

Little-Endian (ARM Cortex, x86): LSB (DD) auf kleinster Adresse. (Speicher: DD CC BB AA)

Big-Endian (Netzwerkprotokolle): MSB (AA) auf kleinster Adresse. (Speicher: AA BB CC DD)

4. Hardwarenahe Programmierung

■ Direkter Register-Zugriff & volatile

Um Hardware-Register zu beschreiben, muss eine feste Hex-Adresse gecastet werden. Das Schlüsselwort `volatile` ist zwingend nötig: Es verbietet dem Compiler, Lese-/Schreibzugriffe wegzuroptimieren oder in Registern zwischenspeichern.

```
// Lese-/Schreibregister (z.B. GPIO Output)
*((volatile uint32_t *)0x40001000) = 0xFFFF;

// KLAUSURFALLE: Read-Only Hardware Register (z.B. ADC-Wert, Status-Flags)
// Muss als 'const volatile' deklariert werden!
// const: Software darf nicht schreiben. volatile: Wert kann sich durch HW ändern.
uint32_t adc_val = *((const volatile uint32_t *)0x40001004);
```

■ Bitmanipulation (Bitmasking)

Einzelne Bits verändern, ohne andere zu beeinflussen (N = Bit-Position, 0 = LSB).

Setzen (1): `REG |= (1 << N);`

Löschen (0): `REG &= (1 << N);`

Umschalten (Toggle): `REG ^= (1 << N);`

Prüfen: `if (REG & (1 << N)) { ... }`

5. Präprozessor & Makros

Der Präprozessor führt reine Text-Ersetzungen aus.

Include Guards: Verhindern, dass Header mehrfach inkludiert werden (Vermeidung von Redefinition-Errors).

```
#ifndef MY_HEADER_H
#define MY_HEADER_H
// Deklarationen...
#endif
```

KLAUSURFALLE Makro-Klammerung: Parameter und das gesamte Makro immer klammern!

```
#define MULT_FALSCH(a, b) a * b
#define MULT_RICHTIG(a, b) ((a) * (b))
int res = MULT_FALSCH(2+3, 4); // Ergibt 2 + 3 * 4 = 14 (FALSCH)
```

6. Interrupts, Nebenläufigkeit & RTOS

■ Interrupt Service Routinen (ISR)

Unterbrechen das Hauptprogramm asynchron bei Hardware-Events. Regeln für ISRs:

Kurz und schnell ausführen.

Keine blockierenden Aufrufe (kein `printf`, `malloc`, `delay`, Mutex blockieren).

Globale Variablen, die zwischen ISR und `main()` geteilt werden, MÜSSEN als `volatile` deklariert sein. Oft muss der Zugriff in `main()` durch kurzzeitiges Deaktivieren der Interrupts atomar gemacht werden.

■ RTOS, Semaphoren & Mutexe

Schützen kritische Bereiche vor gleichzeitigem Zugriff (Data Races).

Mutex: Sperrt eine Ressource (Besitzer-Konzept). Task A sperrt, Task A muss entsperren.

Semaphore: Reiner Zähler. Eignet sich zur Signalisierung zwischen Task und ISR (ISR gibt Semaphore frei, Task konsumiert).

Priority Inversion: High-Prio-Task wartet auf Low-Prio-Task, der einen Mutex hält. Medium-Prio-Task unterbricht Low-Prio-Task. High-Prio-Task verhungert. (Lösung: Priority Inheritance).

Deadlock: Tasks blockieren sich gegenseitig zirkulär (A wartet auf B, B wartet auf A).

7. Watchdog Timer (WDT)

Ein Hardware-Timer, der das System (Microcontroller) automatisch neu startet (Reset), wenn er nicht regelmäßig von der Software zurückgesetzt wird ("kicking" / "feeding the dog").

Zweck: System-Recovery bei Software-Hängern (Endlosschleifen, Deadlocks) oder starken EMI-Störungen, die den Program-Counter verfälschen.

Regel: Der WDT darf niemals in einem Interrupt gefüttert werden (da Interrupts weiterlaufen könnten, auch wenn die main() in einem Deadlock hängt).

8. Wichtige Standard-Funktionen (I/O)

■ printf() & Formatierung

printf benötigt viel Flash und Laufzeit. In Embedded oft per Retargeting auf UART umgeleitet.

Format-Specifier:

- %d / %i: Vorzeichenbehaftet (int)
- %u: Vorzeichenlos (unsigned)
- %x / %X: Hexadezimal -> Sehr wichtig für Register!
- %s: String / %c: Char / %p: Pointer

Formatierung (Klausur-Klassiker):

%04X: Hex-Wert mit führenden Nullen auf exakt 4 Stellen auffüllen (z.B. 0x00FF).

%8d: Integer rechtsbündig, Breite 8 (mit Leerzeichen aufgefüllt).

%.2f: Float auf 2 Nachkommastellen runden.

Code Beispiele

■ Pointer

```
// Adresse printen:
printf("Adresse %p\n", &a);

// Inhalt des Pointers printen (nicht so wichtig)
printf("Inhalt pointer p: %p\n", p);

// Zugriff auf das Element auf das der Pointer //zeigt printen
printf("Element auf das Pointer zeigt: %d\n", *p);

// setzen eines wertes an einer Adresse
*((volatile uint16_t*)0x00F3B404) = 0xFFFF;
// oder
volatile uint8_t *p = (volatile uint8_t *)ADRESS;
*p = 1;
```

■ Makefile

```
#####
# Makefile for TI-Jukebox Project
# Embedded Systems Class
#
# Project: TI-Jukebox
# Class: Embedded Systems
# Author: Felix Miller
# Contact: felix.miller@hm.edu
# Date: 2024-11
#
# Description:
# This Makefile is used for the TI-Jukebox to enable both a build on the host
# machine and for the target MCU.
# Note that this Makefile makes limited use of variables to increase readability
# and tries to avoid cryptic symbols on purpose.
#
# Usage:
# - make main : Build for the host
# - make TARGET=mcu firmware.elf : Build the Firmware elf
# - make TARGET=mcu firmware.bin : Build the Firmware binary
# - make TARGET=mcu flash : Flash to the device
#####

ifeq ($(TARGET),mcu)
    CC = riscv64-unknown-elf-gcc
    CFLAGS = -DMCU_TARGET -DDISP_I2C --specs=picolibc.specs -std=c11 -Wall -Wextra -Wundef -Wshadow -pedantic -
Wdouble-promotion -g -ffreestanding -fno-builtin-printf -fno-common -march=rv32imc -mabi=ilp32 -Os -ffunction-
sections -fdata-sections
else
    CC = gcc
    CFLAGS = -DHOST_TARGET -DDISP_I2C -std=c11 -Wall -Wextra -Wundef -Wshadow -pedantic -Wdouble-promotion -g
endif

# Define the port for flashing
PORT = /dev/ttyUSB0

# Default target
default: main

# Linking for host build
main: main.o playlist.o disp.o i2c_mock.o song_src.o system_mock.o
    $(CC) main.o playlist.o disp.o i2c_mock.o song_src.o system_mock.o -o main

# Linking for device build using the esp32c3 linker script
firmware.elf: main.o playlist.o songs.o disp.o system.o
    $(CC) -T esp32c3.ld -nostdlib -nostartfiles -march=rv32imc -mabi=ilp32 -Wl,--gc-sections -Wl,-
Map=firmware.map main.o playlist.o songs.o disp.o system.o -L. -li2c -o firmware.elf
```

```

# Compile each .c file to an object file
%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

# Compile assembly with song database (this mimics our "sdcard" for now)
songs.o: songs.S
    $(CC) $(CFLAGS) -c $< -o $@

# Create a binary from the ELF file
firmware.bin: firmware.elf
    esptool --chip esp32c3 elf2image --version 2 --dont-append-digest firmware.elf -o firmware.bin

# Flash the binary to the chip
flash: firmware.bin
    esptool --chip esp32c3 -p $(PORT) -b 460800 --before=default_reset --after=hard_reset write_flash --force --flash_mode dio --flash_freq 80m --flash_size 4MB 0x0 firmware.bin

# Unit Tests
jukebox_test.o: test/jukebox_test.c
    $(CC) $(CFLAGS) -I. -c test/jukebox_test.c -o jukebox_test.o

jukebox_test: jukebox_test.o disp.o i2c_mock.o
    $(CC) jukebox_test.o disp.o i2c_mock.o -o jukebox_test -lcmocka

test: jukebox_test

# Clean up the build files
clean:
    rm -f *.o firmware.elf firmware.bin firmware.map main jukebox_test *.bmp

.PHONY: default clean flash test

```

■ Linker Skripte

```

MEMORY {
    icache (rwx) : ORIGIN = 0x4037c000, LENGTH = 16k
    iram   (rwx) : ORIGIN = 0x40380400, LENGTH = 32k
    dram   (rw)  : ORIGIN = 0x3fc80000 + LENGTH(iram), LENGTH = 128k
}

```

```

_eram = ORIGIN(dram) + LENGTH(dram);
ENTRY(_reset)

```

```

SECTIONS {
    .text : { *(.text) *(.text*) } > iram

```

```

    .data : {
        . = ALIGN(16);
        _sdata = .;
        *(.data)
        *(.data*)
        *(.sdata)
        *(.sdata*)
        *(.srodata)
        *(.srodata*)
        *(.rodata)
        *(.rodata*)
        *(.gnu.linkonce.r.*)
        *(.rodata1)
        *(.riscv.*)
        . = ALIGN(16);
        _edata = .;
    } > dram

```

```

    .bss (NOLOAD): {
        . = ALIGN(16);
        _sbss = .;
        *(.bss)
        *(.bss*)
        *(.sbss)
    }

```

```

    *(.sbss*)
    *(COMMON)
    . = ALIGN(16);
    _ebss = .;
} > dram

. = ALIGN(16);
PROVIDE(end = .);
PROVIDE(_end = .);
}

PROVIDE(memset = 0x40000354);
PROVIDE(memcpy = 0x40000358);
PROVIDE(memmove = 0x4000035c);
PROVIDE(memcmp = 0x40000360);
PROVIDE(strcpy = 0x40000364);
PROVIDE(strncpy = 0x40000368);
PROVIDE(strcmp = 0x4000036c);
PROVIDE(strncmp = 0x40000370);
PROVIDE(strlen = 0x40000374);
PROVIDE(strstr = 0x40000378);
PROVIDE(bzero = 0x4000037c);
PROVIDE(isalnum = 0x40000388);
PROVIDE(isalpha = 0x4000038c);
PROVIDE(isspace = 0x40000390);
PROVIDE(isblank = 0x40000394);
PROVIDE(iscntrl = 0x40000398);
PROVIDE(isdigit = 0x4000039c);
PROVIDE(islower = 0x400003a0);
PROVIDE(isgraph = 0x400003a4);
PROVIDE(isprint = 0x400003a8);
PROVIDE(ispunct = 0x400003ac);
PROVIDE(isspace = 0x400003b0);
PROVIDE(isupper = 0x400003b4);
PROVIDE(toupper = 0x400003b8);
PROVIDE(tolower = 0x400003bc);
PROVIDE(toascii = 0x400003c0);
PROVIDE(memccpy = 0x400003c4);
PROVIDE(memchr = 0x400003c8);
PROVIDE(memrchr = 0x400003cc);
PROVIDE(strcasecmp = 0x400003d0);
PROVIDE(strcasestr = 0x400003d4);
PROVIDE(strcat = 0x400003d8);
PROVIDE(strdup = 0x400003dc);
PROVIDE(strchr = 0x400003e0);
PROVIDE(strcspn = 0x400003e4);
PROVIDE(strcoll = 0x400003e8);
PROVIDE(strlcat = 0x400003ec);
PROVIDE(strlcpy = 0x400003f0);
PROVIDE(strlwr = 0x400003f4);
PROVIDE(strncasecmp = 0x400003f8);
PROVIDE(strncat = 0x400003fc);
PROVIDE(strndup = 0x40000400);
PROVIDE(strnlen = 0x40000404);
PROVIDE(strrchr = 0x40000408);
PROVIDE(strsep = 0x4000040c);
PROVIDE(strspn = 0x40000410);
PROVIDE(strtok_r = 0x40000414);
PROVIDE(strupr = 0x40000418);
PROVIDE(longjmp = 0x4000041c);
PROVIDE(setjmp = 0x40000420);
PROVIDE(abs = 0x40000424);
PROVIDE(div = 0x40000428);
PROVIDE(labs = 0x4000042c);
PROVIDE(ldiv = 0x40000430);
PROVIDE(qsort = 0x40000434);
PROVIDE(rand_r = 0x40000438);
PROVIDE(rand = 0x4000043c);
PROVIDE(srand = 0x40000440);
PROVIDE(utoa = 0x40000444);
PROVIDE(itoa = 0x40000448);

```

```

PROVIDE(atoi = 0x4000044c);
PROVIDE(atol = 0x40000450);
PROVIDE(strtol = 0x40000454);
PROVIDE(strtoul = 0x40000458);
PROVIDE(sprintf = 0x40000460);
PROVIDE(__udivdi3 = 0x400008ac);
PROVIDE(__umoddi3 = 0x400008bc);
PROVIDE(__sprintf_r = 0x40000480);
PROVIDE(_fprintf_r = 0x40000484);
PROVIDE(_fprintf_r = 0x40000488);
PROVIDE(_printf_common = 0x4000048c);
PROVIDE(_printf_i = 0x40000490);
PROVIDE(_vfprintf_r = 0x40000494);
PROVIDE(_vfprintf_r = 0x40000498);
PROVIDE(fiprintf = 0x4000049c);
PROVIDE(fprintf = 0x400004a0);
/* The ESP32-C3 ROM exposes a second printf entry at 0x400004a4. We deliberately
   hide it (prefix with XXX) so that the simpler ROM printf at 0x400004a0 above
   stays the single authoritative resolution for the `printf` symbol. */
PROVIDE(XXXprintf = 0x400004a4);
PROVIDE(vfprintf = 0x400004a8);
PROVIDE(vfprintf = 0x400004ac);
PROVIDE(uart_tx_one_char = 0x40000068);
PROVIDE(uart_tx_one_char2 = 0x4000006c);
PROVIDE(uart_rx_one_char = 0x40000070);
PROVIDE(uart_rx_one_char_block = 0x40000074);
PROVIDE(uart_rx_readbuff = 0x40000078);
PROVIDE(__divdi3 = 0x400007b4);
PROVIDE(ets_get_cpu_frequency = 0x40000584);

```

■ Linked Lists

```

#include "playlist.h"

typedef struct playlist_node_s {
    uint32_t id;
    char artist[MAX_ARTIST_LENGTH];
    char title[MAX_TITLE_LENGTH];
    struct playlist_node_s *next;
}node_t;

int insert_at_end(node_t *head, node_t *new_node)
{
    if ( head == NULL || new_node == NULL )
        return -1;

    node_t *current = head;
    while (current->next != NULL)
    {
        current = current->next;
    }
    current->next = new_node;
    new_node->next = NULL;
    return 0;
}

int insert_after_pos(node_t *head, node_t *new_node, size_t pos)
{
    if (head == NULL || new_node == NULL)
        return -1;

    node_t *current = head;

    size_t cnt = 0;
    while (current != NULL)
    {
        if (pos == cnt++)
        {
            new_node->next = current->next;
            current->next = new_node;
            return 0;
        }
    }
}

```

```

    }
    current = current->next;
}
return -1;
}

int count_elements(node_t *head)
{
    if (head == NULL)
        return 0;

    node_t *current = head;

    size_t cnt = 0;
    while (current != 0)
    {
        cnt ++;
        current = current->next;
    }
    return (int)cnt;
}

node_t *get_element(node_t *head, size_t pos)
{
    if (!head)
        return NULL;

    node_t *current = head;
    size_t cnt = 0;
    while (current != 0)
    {
        if (pos == cnt++)
        {
            return current;
        }
        current = current->next;
    }
    return NULL;
}

void print_playlist(node_t *head)
{
    node_t *current = head;
    int ctr = 0;
    while (current != NULL)
    {
        // printf("%p : %p ", current, current->next);
        printf("%d: ", ctr);
        printf("0x%X\n", current->id);
        // printf("Artist: %s\n", current->artist);
        // printf("Song: %s\n\n", current->title);
        current = current->next;
        ctr++;
    }
}

int remove_by_id(node_t *head, uint32_t id)
{
    if (head == NULL)
        return -1;

    node_t *current = head;
    node_t *previous = NULL;

    while (current != NULL)
    {
        if (current->id == id)
        {
            if (previous == NULL)
            {
                // The node to be removed is the head
            }
        }
    }
}

```

```

        // continue as there might be another song with that id that could be removed
        // continue;
        // must return as test playlist_remove_by_id_first expects that
        return -1;
    }
    else
    {
        // The node to be removed is not the head
        previous->next = current->next;
    }
    free(current);
    return 0;
}
previous = current;
current = current->next;
}
// Song with the given ID not found in the playlist
return -1;
}

int remove_by_pos(node_t *head, size_t pos)
{
    if (!head)
        return -1;

    node_t *current = head;
    node_t *previous = head;

    size_t cnt = 0;

    while (current != 0)
    {
        if (pos == cnt)
        {
            previous->next = current->next;
            free(current);
            return 0;
        }
        previous = current;
        current = previous->next;
        cnt++;
    }
    return -1;
}

int move_up(node_t *head, size_t pos)
{
    // works for all pos >= 2 (in other cases head will have to be changed)
    // .next --> node
    // pos-2 --> pos
    // pos --> pos-1
    // pos-1 --> pos+1

    if ( head == NULL )
        return -1;

    if ( pos < 2 )
        return -1;

    //TODO wtf is that, use loop and for pos... use array??
    node_t* pos_n2 = get_element(head, pos-2);
    if ( pos_n2 == NULL )
        return -1;
    node_t* pos_n1 = pos_n2->next;
    if ( pos_n1 == NULL )
        return -1;
    node_t* pos_00 = pos_n1->next;
    if ( pos_00 == NULL )
        return -1;

```



```
node_t* pos_p1 = pos_00->next;

pos_n2->next = pos_00;
pos_00->next = pos_n1;
pos_n1->next = pos_p1;

return 0;
}

int move_down(node_t *head, size_t pos)
{
    // just use move_up
    return move_up(head, pos+1 );
}
```